APUNTES UNIVERSITARIOS

Foundations of Machine Learning and Deep Learning

Apuntes del Tema 11: Attention Mechanisms and Transformers

19 de octubre de 2026

Índice general

1 Apuntes del Tema 11: Attention Mechanisms and Transformers **2**

1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq) 2

1.1.1 1.1 El Enfoque Seq2Seq Tradicional (RNNs) 2

1.1.2 1.2 El Cuello de Botella del Vector de Contexto (*Context Bottleneck*) . . 2

1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014) 2

1.2.1 2.1 Filosofía de la Atención 2

1.2.2 2.2 Formulación Matemática de la Atención de Bahdanau 2

1.2.3 2.3 Inconvenientes de la Atención sobre RNNs 3

1.3 3. Autoatención (*Self-Attention*) en Transformers 3

1.3.1 3.1 Queries, Keys y Values (Q, K, V) 3

1.3.2 3.2 Formulación Matricial: Scaled Dot-Product Attention 4

1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*) 4

1.4.1 4.1 Motivación 4

1.4.2 4.2 Ecuaciones de Multi-Head Attention 4

1.4.3 4.3 Dimensiones de Tensores en Práctica (Batching) 4

1.5 5. Arquitectura del Transformer: Encoder 5

1.5.1 5.1 Codificación Posicional (*Positional Encoding*) 5

1.5.2 5.2 Conexiones Residuales y Normalización por Capa (Add & Norm) . . 5

1.5.3 5.3 Red Feedforward (FFN) 6

1.6 6. Arquitectura del Transformer: Decoder 6

1.6.1 6.1 Masked Multi-Head Self-Attention (Atención Causal) 6

1.6.2 6.2 Cross-Attention (Atención Cruzada Encoder-Decoder) 6

1.6.3 6.3 Capas Finales y Proyección de Salida 6

1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas 7

1.7.1 7.1 Estrategias de Entrenamiento 7

1.7.2 7.2 Estrategias de Decodificación en Inferencia 7

1.7.3 7.3 Evolución de las Arquitecturas Transformer 7

1.8 8. Práctica en Código (PyTorch) 7

1 Apuntes del Tema 11: Attention Mechanisms and Transformers

1.1 1. Introducción y Limitaciones de los Modelos Secuenciales (Seq2Seq)

1.1.1 1.1 El Enfoque Seq2Seq Tradicional (RNNs)

- **Modelado Secuencial:** Las redes neuronales recurrentes (RNNs) procesan las secuencias token a token de forma secuencial.
- **Arquitectura Encoder-Decoder:** En tareas como la traducción automática (p. ej., de inglés a español), el *Encoder* procesa la frase de entrada y condensa su información en un vector de estado oculto final h_{T_x}. Luego, el *Decoder* genera la secuencia traducida token a token condicionado en este vector.

```
Entrada (Inglés)    [ ENCODER (RNN) ]    Vector de Estado H_Tx    [ DECODER (RNN) ]
    Salida (Español)
```

1.1.2 1.2 El Cuello de Botella del Vector de Contexto (*Context Bottleneck*)

- **Limitación Principal:** Comprimir toda una frase de longitud variable en un único vector de tamaño fijo h_{T_x} genera una gran pérdida de información.
- **Fallo en Secuencias Largas:** Aunque funciona bien para frases cortas (p. ej., *"The black cat"* $\rightarrow$ *"El gato negro"*), el modelo falla en oraciones largas y complejas.
- **Dependencias a Largo Plazo:** Las palabras iniciales se "olvidan" a medida que la secuencia crece. Además, no gestiona eficientemente el reordenamiento de palabras entre idiomas.

1.2 2. Mecanismos de Atención en RNNs (Bahdanau et al., 2014)

1.2.1 2.1 Filosofía de la Atención

En lugar de depender de un único vector fijo, la atención permite al *Decoder* acceder a **todos** los estados ocultos del *Encoder* $\{h_1, h_2, \ldots, h_{T_x}\}$ en cada paso de decodificación.

- **Adaptabilidad:** En cada paso t, el *Decoder* calcula qué partes de la frase de entrada son más relevantes para predecir el siguiente token.
- **Vector de Contexto Dinámico (c_t):** Se calcula como una suma ponderada de todos los estados del *Encoder*:

$$c_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i$$

1.2.2 2.2 Formulación Matemática de la Atención de Bahdanau

Para cada paso de tiempo del decodificador i con estado anterior s_{i-1}:

1. **Puntuación de Alineamiento (*Score*):**

$$e_{ij} = a(s_{i-1}, h_j)$$

(donde a es una red feedforward).

1. **Pesos de Atención (Softmax):**

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}$$

1. **Vector de Contexto:**

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

1. **Actualización del Estado y Predicción:**

$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

$$P(y_i \mid y_{1:i-1}, x) = g(y_{i-1}, s_i, c_i)$$

1.2.3 2.3 Inconvenientes de la Atención sobre RNNs

- **Dependencia Recurrente:** La computación sigue siendo secuencial (paso a paso), lo que impide la paralelizabilidad masiva en GPUs durante el entrenamiento.
- **Poco Directo:** La atención solo conecta el *Decoder* con el *Encoder*, pero la información dentro de cada bloque sigue fluyendo de forma secuencial implícita.

1.3 3. Autoatención (*Self-Attention*) en Transformers

Propuesto por Vaswani et al. (2017) en *"Attention is All You Need"*, elimina por completo la recurrencia. Cada palabra puede atender directamente a cualquier otra palabra de la secuencia sin importar la distancia.

1.3.1 3.1 Queries, Keys y Values (Q, K, V)

Para cada token con embedding $x \in \mathbb{R}^d$, se calculan tres proyecciones mediante matrices de pesos entrenables:

1. **Query ($q = W_Q x \in \mathbb{R}^{d_k}$):** Lo que el token busca saber sobre otros.
1. **Key ($k = W_K x \in \mathbb{R}^{d_k}$):** Cómo se identifica o etiqueta el token ante los demás.
1. **Value ($v = W_V x \in \mathbb{R}^{d_v}$):** La información que el token lleva para compartir.

1.3.2 3.2 Formulación Matricial: Scaled Dot-Product Attention

Dada la matriz de entrada $X \in \mathbb{R}^{T_x \times d_{\text{model}}}$, se proyectan las matrices $Q = XW_Q$, $K = XW_K$ y $V = XW_V$.

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- **Matriz de Similitud ($S = \frac{QK^\top}{\sqrt{d_k}}$):** Mide la afinidad entre las consultas y las claves.
- **Factor de Escala ($\sqrt{d_k}$):** Previene que el producto escalar alcance magnitudes muy grandes que saturarían la función Softmax (provocando gradientes nulos).

```
1
2  Entrada X (T x d_model)
3
4        Q = X·W_Q (T x d_k)
5
6        K = X·W_K (T x d_k)      [ Q·K / √d_k ]    [ Softmax ]
7
8        V = X·W_V (T x d_v)                            Salida Z (T x d_v)
```

1.4 4. Atención Multicabeza (*Multi-Head Self-Attention*)

1.4.1 4.1 Motivación

Una sola cabeza de atención solo proyecta en un espacio de subespacio único y puede capturar un único tipo de relación (p. ej., sujeto-verbo). El lenguaje requiere capturar múltiples relaciones simultáneamente (sintácticas, semánticas, modificadores).

1.4.2 4.2 Ecuaciones de Multi-Head Attention

Se entrenan h conjuntos independientes de matrices de proyección $\{W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}\}$:

$$\text{head}_i = \text{Attention}\left(Q^{(i)}, K^{(i)}, V^{(i)}\right) = \text{Attention}\left(XW_Q^{(i)},\, XW_K^{(i)},\, XW_V^{(i)}\right)$$

Las salidas de todas las cabezas se concatenan y se proyectan linealmente con $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ para mantener las dimensiones de entrada:

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)W_O$$

1.4.3 4.3 Dimensiones de Tensores en Práctica (Batching)

- Entrada $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ (B: Batch size, T: Longitud de secuencia).
- Proyección y cambio de forma para procesamiento paralelo:

$$Q, K, V \to \mathbb{R}^{B \times h \times T \times d_k}$$

- Salida de la atención por cabeza: $\mathbb{R}^{B \times h \times T \times d_v}$.
- Concatenación y proyección final: $\mathbb{R}^{B \times T \times d_{\text{model}}}$.

1.5 5. Arquitectura del Transformer: Encoder

El *Encoder* mapea la secuencia de entrada en una representación contextualizada manteniendo la dimensión del eje temporal T y la dimensión del embedding d_{model}. Consta de un apilamiento de N capas idénticas ($N = 6$ en la arquitectura original).

```
Entrada (Tokens)    [ Input Embedding ]    (+)    [ Stack de N Capas Encoder ]
    Representación Contextualizada



                                    [ Positional Encoding ]
```

1.5.1 5.1 Codificación Posicional (*Positional Encoding*)

Dado que la autoatención es invariante a permutaciones (trata las palabras como un conjunto no ordenado), se debe añadir explícitamente información de orden.

- **Incorporación:** Se suma directamente al embedding de entrada en el primer nivel: $z_i = x_i + p_{\text{pos}}$.
- **Sinusoidal Encoding (Original Transformer):**

$$PE_{(\text{pos},2i)} = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(\text{pos},2i+1)} = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

- **Learned Positional Embeddings (BERT, GPT):** Tabla de parámetros entrenables $P \in \mathbb{R}^{T_{\max} \times d_{\text{model}}}$.

1.5.2 5.2 Conexiones Residuales y Normalización por Capa (Add & Norm)

Se aplican alrededor de cada subcapa (atención y red feedforward):

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

- **Residual Connections:** Añaden la entrada de la capa a la salida $(x + \text{Sublayer}(x))$, facilitando el flujo del gradiente sin desvanecimiento ni explosión.
- **Layer Normalization:** Normaliza a través de las características d_{model} independientemente para cada token.
- **Variantes de Orden:**
- *Post-LN (Original):* $\text{LN}(x + \text{Sublayer}(x))$.
- *Pre-LN (Moderno - GPT/BERT):* $x + \text{Sublayer}(\text{LN}(x))$, mejora la estabilidad en redes muy profundas.

1.5.3 5.3 Red Feedforward (FFN)

Se aplica de forma independiente a cada token (*position-wise*):

$$\text{FFN}(x) = \text{Activation}(xW_1 + b_1)W_2 + b_2$$

- Expande la dimensión intermedia a d_{ff} (típicamente $d_{ff} = 4 \times d_{\text{model}}$).
- **Funciones de Activación:**
- *ReLU:* $\max(0, x)$ (Transformer original).
- *GELU:* $x \cdot \Phi(x)$ (Modelos modernos como BERT o GPT).

1.6 6. Arquitectura del Transformer: Decoder

El *Decoder* genera la secuencia objetivo token a token en modo autorregresivo.

```

Entrada desplazada    [ Output Embedding + PE ]    [ Masked Self-Attention ]    [
    Cross-Attention ]    [ FFN ]    [ Linear + Softmax ]
```

1.6.1 6.1 Masked Multi-Head Self-Attention (Atención Causal)

Para evitar que el modelo acceda a tokens futuros durante el entrenamiento, se aplica una **máscara causal** sobre la matriz de puntuaciones previa al Softmax:

$$\text{scores}_{\text{masked}} = \text{scores} + M, \quad \text{donde } M_{ij} = \begin{cases} 0 & \text{si } i \geq j \\ -\infty & \text{si } i < j \end{cases}$$

Esto asegura que en la posición t, la predicción solo dependa de los tokens anteriores $y_{<t}$.

1.6.2 6.2 Cross-Attention (Atención Cruzada Encoder-Decoder)

Conecta la representación del *Encoder* con el *Decoder*:

- **Query (Q):** Proviene del *Decoder* ($X_{\text{dec}}W_Q$).
- **Key (K) y Value (V):** Provienen de la salida final del *Encoder* ($H_{\text{enc}}W_K$, $H_{\text{enc}}W_V$).

$$\text{CrossAttention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

1.6.3 6.3 Capas Finales y Proyección de Salida

1. La salida oculta del decodificador $H_{\text{dec}} \in \mathbb{R}^{B \times T_y \times d_{\text{model}}}$ pasa por una capa lineal que proyecta al tamaño del vocabulario V:

$$\text{logits} = H_{\text{dec}}W_{\text{vocab}}^\top + b$$

1. Se aplica **Softmax** para obtener la distribución de probabilidad $P(y_t \mid y_{<t}, x)$.

1.7 7. Consejos de Entrenamiento, Inferencia y Arquitecturas

1.7.1 7.1 Estrategias de Entrenamiento

- **Subword Tokenization:** Uso de algoritmos como BPE (Byte-Pair Encoding) o WordPiece para evitar la explosión de vocabulario y manejar palabras fuera del diccionario (OOV).
- **Padding Masking:** Dado que los batches se rellenan con `<PAD>` para igualar longitudes, se aplica un enmascaramiento con $-\infty$ para evitar atender a posiciones de relleno.
- **Warm-up Scheduling:** Incremento gradual de la tasa de aprendizaje durante los primeros pasos seguido de un decaimiento (p. ej., coseno).

1.7.2 7.2 Estrategias de Decodificación en Inferencia

- **Greedy Decoding:** Selecciona $y_t = \arg\max_v P(v)$. Rápido, pero puede caer en subóptimos locales.
- **Beam Search:** Mantiene las k mejores hipótesis parciales acumulando sus probabilidades logarítmicas.
- **Temperature Scaling:** Ajusta los logits dividiéndolos por T antes del Softmax para controlar la creatividad/aleatoriedad:
- $T < 1$: Distribución más picuda (más determinista).
- $T > 1$: Distribución más plana (más diversa).

1.7.3 7.3 Evolución de las Arquitecturas Transformer

- **Encoder-Only (p. ej., BERT):** Utiliza atención bidireccional. Ideal para tareas de comprensión, clasificación de textos y extracción de entidades.
- **Decoder-Only (p. ej., GPT-3, LLaMA, Mistral):** Utiliza atención causal. Orientado a la generación de texto autorregresiva.
- **Encoder-Decoder (p. ej., T5, BART):** Conserva la arquitectura original seq2seq para traducción y resumen.

1.8 8. Práctica en Código (PyTorch)

A continuación se incluye una implementación ejecutable de un módulo de **Multi-Head Self-Attention** nativo con enmascaramiento causal opcional:

```
1
2  import torch
3
4  import torch.nn as nn
5
6  import torch.nn.functional as F
7
8  import math
9
10
11
```

```
class MultiHeadSelfAttention(nn.Module):

    def __init__(self, d_model=512, num_heads=8):

        super(MultiHeadSelfAttention, self).__init__()

        assert d_model % num_heads == 0, "d_model debe ser divisible por num_heads"



        self.d_model = d_model

        self.num_heads = num_heads

        self.d_k = d_model // num_heads



        # Proyecciones lineales para Q, K, V

        self.W_q = nn.Linear(d_model, d_model, bias=False)

        self.W_k = nn.Linear(d_model, d_model, bias=False)

        self.W_v = nn.Linear(d_model, d_model, bias=False)



        # Proyección de salida

        self.W_o = nn.Linear(d_model, d_model, bias=False)



    def forward(self, x, mask=None):

        batch_size, seq_len, _ = x.size()



        # 1. Proyección lineal -> (B, T, h, d_k) -> Transposición a (B, h, T, d_k)

        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)

        K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)

        V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose
    (1, 2)



        # 2. Scaled Dot-Product Attention

        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)


```

```
67
68          if mask is not None:
69
70              scores = scores.masked_fill(mask == 0, -1e9)
71
72
73
74          attn_weights = F.softmax(scores, dim=-1)
75
76          attn_output = torch.matmul(attn_weights, V) # (B, h, T, d_k)
77
78
79
80          # 3. Concatenación de cabezas y proyección de salida
81
82          attn_output = attn_output.transpose(1, 2).contiguous().view(batch_size,
    seq_len, self.d_model)
83
84          output = self.W_o(attn_output)
85
86
87
88          return output
89
90
91
92  # Ejemplo de prueba rápida:
93
94  if __name__ == "__main__":
95
96      B, T, d_model = 2, 5, 64
97
98      x = torch.randn(B, T, d_model)
99
100
101
102     # Máscara Causal (Triangular Inferior)
103
104     causal_mask = torch.tril(torch.ones(T, T)).unsqueeze(0).unsqueeze(0)
105
106
107
108     mha = MultiHeadSelfAttention(d_model=d_model, num_heads=4)
109
110     out = mha(x, mask=causal_mask)
111
112     print(f"Dimensión del tensor de entrada: {x.shape}")
113
114     print(f"Dimensión del tensor procesado: {out.shape}")
```

Dime si necesitas hacer alguna prueba de autoevaluación o repasar algún detalle específico de las fuentes del máster.